

Deployment & Developer Guide

Connecting a property's own brand website to Uppsolut Stay: live availability, prices, and bookings straight into the front desk.

This guide is for two readers. The **enterprise administrator** who sets the property up for online sale and issues the API key in the Hub, and the **web developer** who builds the property's website against the Website API.

Part I explains what the integration is. Part II is the deployment guide, step by step, with screenshots of every screen involved. Part III is the complete API reference with request and response examples. The appendices list every error code and a go-live checklist.

Document

Website API — Deployment & Developer Guide

API version

v1

Edition

1.2 · September 2026

Audience

Enterprise administrators, web developers

Contents

PART I — OVERVIEW	2
1. What you are building	2
2. Who does what	2
3. How a booking flows	3
PART II — DEPLOYMENT GUIDE	4
4. Before you start	4
5. Step 1 — Configure the property for online sale	4
5.1 Choosing which extras go out	6
6. Step 2 — Create an API key	7
7. Step 3 — Hand the configuration to the developer	9
8. Managing keys afterwards	9
9. Hosting options and payment	10
Server-to-server (recommended)	10
Browser-direct (static sites only)	10
Payment	10
10. Where website bookings appear in Uppsolut Stay	10
PART III — API REFERENCE	12
11. Basics	12
Authentication	12
Dates and "today"	12
Error format	12
12. Endpoints	13
12.1 List properties	13
12.2 Property details	13
12.3 Availability calendar	14
12.4 Quote a stay	15
12.5 Create a booking	17
12.6 Look up a booking	18
13. Good behaviour	18
14. Quick start (Node)	18
APPENDIX A — ERROR CODES	20
APPENDIX B — GO-LIVE CHECKLIST	22
In the Hub	22
On the website	22
Test together	22

PART I

Overview

What the Website API is, who does what, and how a booking made on a property's website becomes a reservation at the front desk.

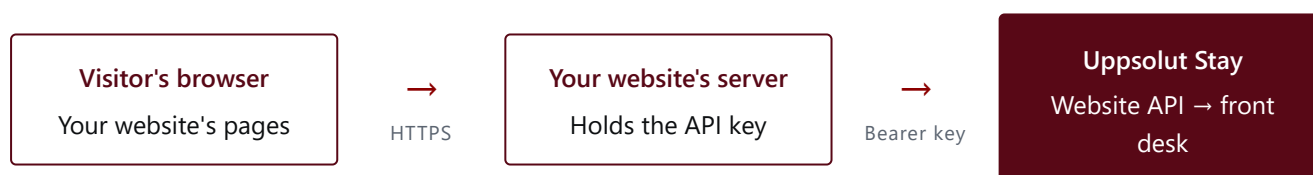
1. What you are building

A property website built on the Website API is a **front** for one property, or for several properties of the same enterprise behind one portal. It has three parts:

1. **Property pages** — name, description, photos, location, contact details, check-in and check-out times, facilities, and each room type with its features.
2. **A booking page** — the visitor picks dates and guests, sees which room types are free and what the stay costs, enters minimal guest details, and confirms.
3. **A "my booking" page** (optional) — confirmation number plus email shows the live status of a stay.

Everything the site shows comes from Uppsolut Stay through the API, so the property maintains one set of facts. Rates and stop-sales set at the desk are what the site sells. A booking made on the site appears at the desk immediately, as an ordinary reservation.

The property brings the design, hosting, domain and any payment handling. Uppsolut Stay provides content, availability, prices and the booking itself.



2. Who does what

Role	Responsibilities	Where
Enterprise administrator	Configures what each property shows and sells online; creates, rotates and revokes API keys; hands the key to the developer.	Hub → Website API
Web developer	Builds the site; stores the key on the server; calls the API for content, availability, quotes and bookings; emails the guest a confirmation.	Your website
Revenue / reservations	Keeps the Price Calendar filled for the plan the site sells; manages stop-sales.	Controls → Revenue, Availability
Front desk	Sees website bookings as normal reservations and honours the policies the site promised.	Reservations

3. How a booking flows

1 The visitor searches

Your site calls the availability endpoint for the chosen dates. The answer is live inventory per room type per night, with stop-sales marked as closed and the nightly room rate on the plan the Hub configured.

2 The visitor picks a room

Your site calls the quote endpoint. The answer is the authoritative total: room rate, extra-person surcharges, package components, taxes, service charge and Green Tax, computed by the same code the desk posts charges with.

3 The visitor confirms

Your site calls the booking endpoint with a fresh `Idempotency-Key`. Uppsolut Stay checks the stay once more, creates a *Reserved* reservation with a confirmation number from the property's own sequence, opens the folio, and records the booking against your key. If the last room went in the meantime, the answer is `SOLD_OUT`. The website can never overbook.

4 Your site confirms to the guest

Show the confirmation number and email it. Uppsolut Stay does not email the guest for website bookings.

KEY PRINCIPLE

There is nothing to synchronise. The site holds no copy of availability or prices; every call reads the live state of the property. Two visitors racing for the last room are resolved at booking time, and the site is designed to handle a `SOLD_OUT` answer gracefully.

PART II

Deployment Guide

For the enterprise administrator. Configuring a property for online sale, issuing an API key, handing it to the developer, and managing keys afterwards. Every step happens in the Hub, the enterprise-wide administration area.

4. Before you start

You need:

- An **enterprise-level** account with the **Integrations** permission. The Hub is not reachable by users pinned to a single property.
- The property **active** in Uppsolut Stay, with its room types and rooms set up.
- A **rate plan** the website will sell, with its **Price Calendar** filled across the period you want to sell. Nights with no price are not sold online.
- A secure channel to hand the key to the developer (a password manager share, not email or chat).

Open the Hub at `/e/<enterprise>/hub` and choose **Website API** in the sidebar. It has two tabs: **API Keys** and **Properties**.

5. Step 1 — Configure the property for online sale

On the **Properties** tab, every active property is listed with its current state: **Bookable**, **No rate plan** or **Booking off**. Choose **Edit** on the property.

Properties

What each property's website shows (headline, description, photos, policies) and sells (the rate plan and meal plan every online booking is made on). A property with no rate plan chosen is listed but cannot be booked online.

Veyo Beach Resort VEYO-MAIN Bookable	Edit
Sells Best Available Rate · min 1 night · up to 365 nights ahead 2 keys · 2 website bookings · no photos	
Veyo Lagoon Retreat VEYO-LAGOON No rate plan Not set up	Edit
Sells — · min 1 night · up to 365 nights ahead 0 keys · 0 website bookings · no photos	

Figure 1. Hub → Website API → Properties. Each property shows what it sells and whether it can be booked online yet. A property with no rate plan chosen is listed by the API but cannot take a booking.

Setting	What it does
Accept bookings from the website	The master switch. When off, the site can still show the property, but the API reports booking as disabled and the site should hide its booking form.
Rate plan to sell	Required for booking. Every website reservation is made on this plan and priced from its Price Calendar, with the property's Base plan as the fallback for a night the plan does not price — the same rule the desk uses. Negotiated (agent-only) plans are not offered.
Meal plan	The plan every website reservation is made on, and the default the guest sees when they may choose.
Let guests choose their meal plan	Off: every booking uses the plan above. On: the site offers this property's active meal plans, with that one preselected.
Offer paid extras online	Off: the site sells rooms only. On: a tick-list appears of everything marked sell separately in Allocations, and you choose which of them go out.
Extras offered through the APIs	The tick-list itself. A ticked extra is on sale online; an unticked one stays available to the front desk but is never offered outside it. A newly created extra starts ticked. See section 5.1.
Minimum stay	Shortest stay the site may sell. A website-only limit; the desk is not bound by it.
Booking window	How many nights ahead the site may look and book. Website-only.
Note for the front desk	Added to the remarks of every website reservation, for example "Website booking: collect balance at check-in".
Headline, Description	Marketing copy the site displays.
Photo URLs	One per line. The first is the main image. Host the files on the website or a CDN; Uppsolut Stay stores the addresses only.
Policies	Cancellation, deposit, children, pets. Shown on the booking page. Write what the desk actually honours; nothing here is enforced by the system.

Veyo Beach Resort — website settings

Shown on the property's website and used for every booking it makes.

Online booking

☐ Accept bookings from the website

Rate plan to sell *

Best Available Rate (BAR)

Every website booking is made on this plan. Negotiated plans are not offered.

☐ Offer paid extras online

Transfers, spa treatments and anything else marked "sell separately" in Allocations. Turn this on and pick which of them go out below.

☐ Let guests choose their meal plan

Off: every booking uses the plan chosen below. On: the site offers this property's active meal plans, with that one preselected.

Extras offered through the APIs

2 of 3

All

☐ Lunch LN
☒ Airport Transfer TRF-AIR
☒ Speedboat Transfer TRF-SB

An unticked extra is still there for the front desk to add — it is simply not offered by the website or the channel manager. A newly created extra starts published.

2 more allocations are part of a rate or meal plan only, so there is nothing to sell on its own.

Meal plan

None

Minimum stay (nights)

1

Booking window (nights ahead)

365

Note for the front desk

Website booking: collect balance at check-in

Added to the remarks of every website reservation.

Figure 2. The property's website settings. What the site shows, and what every booking it makes is priced on. The right-hand panel appears once paid extras are switched on.

NOTE

Until a rate plan is chosen, the property is listed by the API with `booking.enabled: false` and a reason. Nothing is ever sold on a guessed plan.

5.1 Choosing which extras go out

Switching **Offer paid extras online** on reveals a tick-list of everything the property sells on its own. Tick the ones the website and the channel manager may offer; leave the rest for the front desk.

Uppsolut Stay · Website API — Deployment & Developer Guide · v1.2

Page 6 of 23

Extras offered through the APIs

2 of 3

All

☐ Lunch LN
☒ Airport Transfer TRF-AIR
☒ Speedboat Transfer TRF-SB

An unticked extra is still there for the front desk to add — it is simply not offered by the website or the channel manager. A newly created extra starts published.

2 more allocations are part of a rate or meal plan only, so there is nothing to sell on its own.

Figure 3. Extras offered through the APIs. Here the two transfers are on sale online and Lunch is not — the desk can still add it to any booking. The note beneath counts the allocations that are part of a rate or meal plan and so have nothing to sell separately.

The list is not a second catalogue. It is the property's own **Allocations**, from **Revenue** → **Allocations**, where an extra is created, priced and marked *Sell Separate*. Only a sell-separate extra can appear here, because only a sell-separate extra can be bought on its own; the Hub then decides whether that same item is also sold outside the front desk.

Code	Name	Type	Charge Code	Rhythm	Mode	Current Price	Status
BF	Breakfast	F&B	2901	Every night	Add to Rate	A \$10 / C \$5	Active
DN	Dinner	F&B	2903	Every night	Add to Rate	A \$30 / C \$15	Active
LN	Lunch	F&B	2902	Every night	Add to Rate Sell Separate	A \$20 / C \$10	Active
TRF-AIR	Airport Transfer	Transfer	5001	On arrival night	Add to Rate Sell Separate Online	A \$40 / C \$20	Active
TRF-SB	Speedboat Transfer	Transfer	5002	On arrival night	Add to Rate Sell Separate Online	A \$50 / C \$25	Active

Figure 4. Revenue → Allocations. Where the extras come from. *Sell Separate* means the desk can add it to a booking; *Online* means the Hub has published it to the APIs as well. Lunch is sold at the desk only.

WITHDRAWING AN EXTRA

Unticking one takes it off the catalogue immediately and the API then refuses it, so a site holding a stale id is told (`ADD_ON_NOT_FOUND`) rather than quietly billing something different from what the guest agreed to. Read the catalogue on each visit rather than storing ids.

6. Step 2 — Create an API key

On the API Keys tab, choose New key.

API keys

+ New key

One key per website. A key may cover one property or several — the site can only see and book the properties on its list. Only the first characters are kept here; the full key is shown once, when created.

Key	Properties	Origins	Status	Last used	Bookings
Veyo brand website (localhost:4000) wsk_a1c19108... Created 9/6/2026 by Veyo Admin	Veyo Beach Resort	Server only	Active	9/7/2026, 11:08:24 AM	2
www.veyo-beach.test wsk_739fd8f9... Created 9/6/2026 by Veyo Admin	Veyo Beach Resort	Server only	Revoked	9/6/2026, 11:04:13 AM	0

Figure 5. Hub → Website API → API Keys. Only the first characters of each key are kept, as a label. The list shows what each key covers, whether it is server-only, and how much it has been used.

New API key

The key is shown once after you save it — have somewhere safe ready to paste it.

Name *

www.veyo-beach.com

Properties *

☒ Veyo Beach Resort VEYO-MAIN
☐ Veyo Lagoon Retreat VEYO-LAGOON

The website can only see and book the properties ticked here.

Browser origins (optional)

https://www.hotel.com
https://booking.hotel.com

Only needed if the website calls the API directly from the visitor's browser. Leave empty for the recommended server-to-server setup, where the key never leaves the website's server.

Expires (optional)

dd/mm/yyyy

Cancel

Create key

Figure 6. Creating a key. The properties ticked here are the only ones the website can see or book — any other answers "not found". Leave *Browser origins* empty unless the site will call the API from the visitor's browser.

Name	Usually the website's domain, for example <code>www.hotel.com</code> .
Properties	Tick the property, or several for one portal covering a group. The key can only see and book the properties ticked here. Any other property answers "not found".
Browser origins	Leave empty for the recommended server-to-server setup. Fill it in only if the site will call the API directly from the visitor's browser (see section 9).
Expires	Optional. A good practice for keys given to an outside agency.

Save. The key is shown **once**, in a dialog. Copy it before closing.

Uppsolut Stay · Website API — Deployment & Developer Guide · v1.2

Page 8 of 23

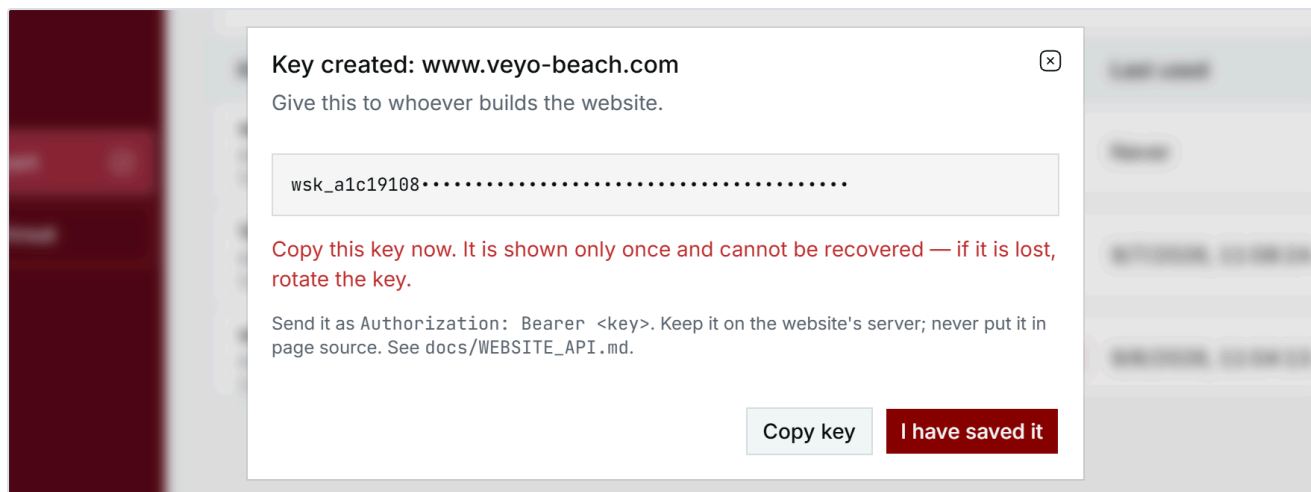


Figure 7. The key, shown once. Masked here for print. Copy it straight into the website's secret store — only a hash is kept, so it cannot be shown again.

IMPORTANT

Only a hash of the key is stored. It cannot be shown again. If it is lost, use **Rotate** to issue a replacement; the old key stops working the moment you do. Keys begin with `wsk_` and are 68 characters long.

7. Step 3 — Hand the configuration to the developer

The website needs three values. They are **server-side secrets**: environment variables or a secrets manager, never page source or a public repository.

```
UPPSOLUT_API_BASE=https://<your-uppsolut-host>/api/website/v1
UPPSOLUT_API_KEY=wsk_...
UPPSOLUT_PROPERTY_ID=<id returned by GET /properties>
```

The developer obtains the property id by calling `GET /properties` with the key; it lists exactly the properties the key covers.

8. Managing keys afterwards

Action	Effect	When to use
Edit	Changes the name, properties, browser origins or expiry. The key itself is unchanged.	A new property joins the portal; a domain changes.
Rotate	Issues a new secret on the same key record. History, properties and booking counts stay attached. The previous secret stops working immediately.	On a schedule; when a developer leaves; when a laptop is lost.

Action	Effect	When to use
Revoke	Permanent. The site loses access immediately. Bookings already made are untouched. The record stays for the audit trail.	The site is decommissioned or the agency contract ends.

The list shows, per key, the properties it covers, whether it is server-only or browser-enabled, its status, when it was last used, and how many bookings it has made. Every creation, rotation, revocation and settings change is written to the enterprise's activity log with the administrator's name.

9. Hosting options and payment

Server-to-server (recommended)

The website's backend calls the Website API and the pages call the backend. The key never reaches a browser. Any stack works: a Next.js or Nuxt server route, a PHP or Laravel controller, a WordPress plugin, a serverless function. Leave **Browser origins** empty in the Hub.

Browser-direct (static sites only)

Add the site's origin(s) to the key, for example `https://www.hotel.com`. The API then answers CORS preflight for those origins. Accept that anyone who reads the page source can read the key and use it to read availability and create reservations. Mitigate by restricting the key to one property, setting an expiry, and rotating regularly. If in doubt, choose server-to-server.

Payment

The API does not take payment. Three patterns, in increasing complexity:

1. **Pay at the property.** Say so in the policies and the desk note. Simplest.
2. **Deposit link after booking.** Book first, then send the guest a payment link from your own provider and keep the reference in your records.
3. **Pay before booking.** Take payment on the site, then create the booking with the payment reference in `remarks`. Handle the rare `SOLD_OUT` after payment with a refund.

Whatever you choose, the desk sees the Hub's note on every website reservation, so nothing is ambiguous at check-in.

10. Where website bookings appear in Uppsolut Stay

Screen	What you see
Reservations	Website bookings look like any other reservation. Remarks begin with "Booked via website (ref WEB-...)". Search by confirmation number or the WEB reference.
Profiles	Guests created by the website are ordinary guest profiles, matched by email on return visits.

Screen	What you see
Hub → Website API → API Keys	Last used, booking count, status per key.
Hub → Website API → Properties	What each property shows and sells online, key count, website booking count.
Activity log	Every key and settings change, by whom, when.
Controls → Revenue → Price Calendar	What the site sells. No website-specific pricing screens; the desk's truth is the site's truth.
Availability → Stop-Sale	Closed dates, which the site shows as closed.

PART III

API Reference

For the web developer. Authentication, conventions, every endpoint with request and response examples, and a quick-start in Node.

11. Basics

Base URL	<code>https://<your-uppsolut-host>/api/website/v1</code>
Format	JSON in, JSON out, UTF-8. Dates are <code>YYYY-MM-DD</code> calendar days with no time zone. Money is a plain number in the property's currency, two decimals.
Authentication	API key in a header on every request. No cookies, no sessions.
Versioning	The <code>v1</code> path segment. Fields may be added to responses at any time; nothing is removed or renamed within <code>v1</code> .
Caching	All responses are <code>Cache-Control: no-store</code> . Cache on your side for at most a minute if you must.

Authentication

Send the key in the `Authorization` header (preferred) or in `X-API-Key`. Never in a URL.

```
Authorization: Bearer wsk_3f9c...
```

Status	Code	Meaning
401	MISSING_API_KEY	No key header present.
401	INVALID_API_KEY	Unknown, revoked or expired. Deliberately the same answer for all three.

Dates and "today"

A property has a **business date**, its operational "today", which moves forward only when the desk runs End-of-Day. `GET /properties/{id}` returns it as `businessDate`. It is the earliest check-in the API accepts and can lag the calendar date by a day. Build the date picker off `businessDate`, not the visitor's clock.

Check-out is exclusive: a stay from 2026-03-10 to 2026-03-12 is two nights.

Error format

Every error is JSON with a stable `code` to switch on and a human-readable `error`. Validation errors add `details` keyed by field path. The full list of codes is in Appendix A.

```
{ "error": "Invalid request.", "code": "VALIDATION",
  "details": { "guest.email": "A valid email is required" } }
```

12. Endpoints

12.1 List properties

GET /properties

Every active property this key may act on.

```
{
  "properties": [
    {
      "id": "8f1c...", "code": "VILLA", "name": "Veyo Island Villas",
      "headline": "Barefoot luxury on a private island",
      "currency": "USD", "timeZone": "Indian/Maldives", "starRating": 5,
      "imageUrl": "https://cdn.hotel.com/hero.jpg",
      "bookingEnabled": true
    }
  ]
}
```

12.2 Property details

GET /properties/{propertyId}

Everything a property page needs.

```
{
  "property": {
    "id": "8f1c...", "code": "VILLA", "name": "Veyo Island Villas",
    "legalName": "Veyo Hospitality Pvt Ltd",
    "headline": "Barefoot luxury on a private island",
    "description": "Long-form marketing copy set in the Hub...",
    "imageUrls": ["https://cdn.hotel.com/hero.jpg", "https://cdn.hotel.com/pool.jpg"],
    "logoUrl": "https://cdn.hotel.com/logo.png", "brandColor": "#0E6F7A", "starRating": 5,
    "address": "Veyo Island, Baa Atoll",
    "location": { "latitude": 5.21, "longitude": 73.05 },
    "contact": { "phone": "+960 660 0000", "email": "stay@veyo.example" },
    "checkInTime": "14:00", "checkOutTime": "11:00",
    "currency": "USD", "timeZone": "Indian/Maldives",
    "pricesIncludeTaxes": true,
    "businessDate": "2026-03-04",
    "policies": "Free cancellation up to 7 days before arrival...",
    "facilities": [{ "name": "Infinity pool", "description": null }],
    "roomTypes": [
      {
        "id": "c2a1...", "code": "BVL", "name": "Beach Villa", "description": "...",
        "baseOccupancy": 2, "maxOccupancy": 3, "totalRooms": 12,
        "features": [
          { "category": "BED_TYPE", "code": "KING", "label": "King bed" },
          { "category": "ROOM_VIEW", "code": "OCEAN", "label": "Ocean view" }
        ]
      }
    ]
  },
  "booking": {
    "enabled": true, "reason": null,

```

```

    "ratePlan": { "id": "...", "code": "BAR", "name": "Best Available Rate" },
    "mealPlanCode": "BB",
    "mealPlanSelectable": true, "mealPlanAffectsPrice": true,
    "mealPlans": [
      { "code": "RO", "name": "Room Only", "isDefault": false },
      { "code": "BB", "name": "Bed & Breakfast", "isDefault": true }
    ],
    "addOns": [
      { "id": "9a2f...", "code": "TRF-SB", "name": "Speedboat Transfer",
        "type": "TRANSFER", "postingRhythm": "ARRIVAL_NIGHT" }
    ],
    "minNights": 1, "maxNightsAhead": 365
  }
}
}

```

- `roomTypes` contains only sellable types. `totalRooms` is informational, not availability.
- `booking.enabled` is false when the Hub has not chosen a rate plan or has switched booking off; `booking.reason` says which. Show the property, hide the form.
- `pricesIncludeTaxes` tells you how to label prices. The quote endpoint always returns the tax breakdown either way.

Meal plans and extras

Both are switched on per property in the Hub, so both lists are often empty. Build the page to cope with that rather than assuming they are there.

- `mealPlanSelectable` is true when the guest may choose; `mealPlans` then lists the property's active plans with the default flagged.
- `mealPlanAffectsPrice` says whether choosing a different plan changes the total. Some properties price per person off the meal plan; others carry the price in the rate plan and treat the meal plan as a label. Do not imply a cost that is not there.
- `addOns` are the paid extras the property sells online. **They carry no price**, on purpose: what one costs depends on the party, the length of stay, and whether it is charged every night or once on arrival. Send the ids to the quote and show the line it returns.
- The catalogue is the property's own choice, extra by extra, and can change — **read it rather than hard-coding ids**. An extra that has been withdrawn is refused by the quote as `ADD_ON_NOT_FOUND`, never silently dropped.

12.3 Availability calendar

GET `/properties/{propertyId}/availability?from=2026-03-10&to=2026-03-15`

Per room type, per **night** in `[from, to)`. At most 62 nights per call.

```

{
  "propertyId": "8f1c...", "from": "2026-03-10", "to": "2026-03-15", "nights": 5,
  "currency": "USD", "pricesIncludeTaxes": true,
  "ratePlan": { "id": "...", "code": "BAR", "name": "Best Available Rate" },
  "bookingEnabled": true,
  "roomTypes": [
    {

```

```

    "id": "c2a1...", "code": "BVL", "name": "Beach Villa",
    "baseOccupancy": 2, "maxOccupancy": 3,
    "nights": [
      { "date": "2026-03-10", "available": 4, "closed": false, "price": 450,
        "extraAdultPrice": 90, "extraChildPrice": 40 },
      { "date": "2026-03-11", "available": 0, "closed": true, "price": 450,
        "extraAdultPrice": 90, "extraChildPrice": 40 },
      { "date": "2026-03-12", "available": 2, "closed": false, "price": null,
        "extraAdultPrice": null, "extraChildPrice": null }
    ],
    "minAvailable": 0, "bookable": false, "roomRateTotal": null
  }
]
}

```

Field	Meaning
<code>available</code>	Rooms of the type free that night. Real inventory: out-of-order rooms and group-block holds are already excluded. Never below 0.
<code>closed</code>	A stop-sale applies. Treat the night as unbookable regardless of <code>available</code> .
<code>price</code>	The room rate for that night only on the configured plan, before tax handling. <code>null</code> means no price is set; the stay cannot be booked online through it.
<code>minAvailable</code>	Lowest <code>available</code> across the window: how many rooms could be booked for every night.
<code>bookable</code>	Every night open, in stock and priced, and booking enabled.
<code>roomRateTotal</code>	Sum of nightly room rates when every night is priced. Room rate only.

DO NOT ADD UP NIGHTLY PRICES

For the total a guest pays, call the quote endpoint. It adds taxes, service charge, Green Tax and package components. The calendar figure is the bare room rate.

Typical use: a date picker calls this once for the chosen range, or once per month to grey out sold-out days, then a results page shows one card per room type using `bookable`, `minAvailable` and `roomRateTotal`.

12.4 Quote a stay

POST `/properties/{propertyId}/quote`

```

{ "checkIn": "2026-03-10", "checkOut": "2026-03-12",
  "roomTypeId": "c2a1...", "adults": 2, "children": 1,
  "mealPlanCode": "BB",
  "addOnIds": ["9a2f..."] }

```

The **authoritative price** for a stay, computed by exactly the code Uppsolut Stay posts charges with. Never writes anything.


```
{
  "quote": {
    "propertyId": "8f1c...",
    "roomType": { "id": "c2a1...", "code": "BVL", "name": "Beach Villa" },
    "ratePlan": { "id": "...", "code": "BAR", "name": "Best Available Rate" },
    "mealPlanCode": "BB",
    "checkIn": "2026-03-10", "checkOut": "2026-03-12", "nights": 2,
    "adults": 2, "children": 1,
    "currency": "USD", "pricesIncludeTaxes": true,
    "available": true, "roomsAvailable": 4,
    "totals": {
      "roomBase": 900, "extraOccupancy": 80, "packageAllocations": 0,
      "taxes": 156.8, "greenTax": 18, "grandTotal": 1154.8
    },
    "allocations": [
      { "id": "...", "code": "BF", "name": "Breakfast", "source": "MEAL_PLAN",
        "mode": "ADD_TO_RATE", "amount": 96 },
      { "id": "9a2f...", "code": "TRF-SB", "name": "Speedboat Transfer",
        "source": "MANUAL", "mode": "ADD_TO_RATE", "amount": 174 }
    ],
    "taxLines": [ { "name": "TGST", "ratePercent": 16, "amount": 156.8 } ],
    "nightly": [
      { "date": "2026-03-10", "rate": 450, "roomCharge": 490, "taxes": 87.4, "total": 577.4 },
      { "date": "2026-03-11", "rate": 450, "roomCharge": 490, "taxes": 87.4, "total": 577.4 }
    ],
    "warnings": []
  }
}
```

- `available` is false when the stay cannot be booked right now: sold out, a closed night, or an unpriced night. Show the price if you like, but disable the button.
- `totals.grandTotal` is what to display as the price of the stay. `taxLines` and `greenTax` give the breakdown.
- `warnings` are informational strings. Log them; do not show them raw to guests.
- `allocations` itemises everything on the stay. `source` says why each line is there: `RATE_PLAN` or `MEAL_PLAN` came with what the guest chose, so show it as included; `MANUAL` means they ticked it, so show it as removable.
- `mode` matters for wording. `INCLUDE_IN_RATE` is carved out of the room line, so its amount is **already inside** `roomBase` and must not be added again. `ADD_TO_RATE` sits on top. `grandTotal` is correct either way.

BOTH FIELDS ARE OPTIONAL

Omit `mealPlanCode` and the booking uses the property's default; omit `addOnIds` and no extras are added. Send either to a property that does not offer it and the request is *refused* rather than silently ignored, so a quote can never disagree with the booking that follows it.

The stay is validated the same way a booking is: `MIN_STAY`, `TOO_FAR_AHEAD`, `INVALID_OCCUPANCY` and the other codes in Appendix A apply.

12.5 Create a booking

POST /properties/{propertyId}/bookings

Idempotency-Key: 6d0b2f9e-1c3a-4e5f-9a7b-...

```
{
  "checkIn": "2026-03-10", "checkOut": "2026-03-12",
  "roomTypeId": "c2a1...", "adults": 2, "children": 1,
  "mealPlanCode": "BB",
  "addOnIds": ["9a2f..."],
  "guest": {
    "firstName": "Ada", "lastName": "Lovelace",
    "email": "ada@example.com", "phone": "+44 7700 900000"
  },
  "remarks": "Arriving on the late seaplane"
}
```

Creates a real reservation with status **RESERVED**, a confirmation number from the property's own sequence, and a master folio, exactly what the front desk would create. The guest is matched to an existing profile by email (case-insensitive) or a new profile is created. Only **firstName** and **email** are required.

IDEMPOTENCY

Generate a fresh UUID per booking attempt and send it as **Idempotency-Key** (or as **idempotencyKey** in the body). If the request times out or returns a 5xx, retry with the **same** key: you get the same booking back with **replayed: true** and status 200, never a second reservation. Keys are scoped to your API key.

Response **201 Created**, or **200 OK** on a replay:

```
{
  "booking": {
    "bookingId": "4c1e...",
    "confirmationNo": "VILLA-000482",
    "reservationId": "...",
    "status": "RESERVED",
    "replayed": false,
    "property": { "id": "8f1c...", "name": "Veyo Island Villas",
      "checkInTime": "14:00", "checkOutTime": "11:00" },
    "guest": { "firstName": "Ada", "lastName": "Lovelace", "email": "ada@example.com" },
    "quote": { "...": "same shape as the quote endpoint – the confirmed figures" }
  }
}
```

Show **confirmationNo** on the confirmation page and in your email to the guest. The desk can find the reservation by that number or by the **WEB-xxxxxxx** reference in its remarks.

Refusals, all **409**: **SOLD_OUT**, **STOP_SALE**, **NO_RATE**, **BOOKING_DISABLED**. If the last room went between the quote and the booking you get **SOLD_OUT**; send the guest back to the dates step.

12.6 Look up a booking

```
GET /bookings/{confirmationNo}?email=ada@example.com
```

A "manage my booking" lookup. Only bookings made through **this key** are visible, and the email must match the one used to book. Returns the same `booking` object as 12.5 with the live reservation `status`: `RESERVED`, `IN_HOUSE`, `CHECKED_OUT`, `NO_SHOW` or `CANCELLED`.

There is no cancel endpoint in v1. Cancellations go through the property, which is what the policies text should say.

13. Good behaviour

There is no hard rate limit today. Please:

- call `availability` once per search, not once per day of a calendar;
- cache property details for a few minutes; they change rarely;
- always send `Idempotency-Key` on bookings and retry with it, never without it;
- never call the API from a browser unless the key is restricted to your origins and you accept that it is public.

14. Quick start (Node)

```
const BASE = process.env.UPPSOLUT_API_BASE; // https://stay.example.com/api/website/v1
const KEY = process.env.UPPSOLUT_API_KEY; // wsk_...
const PROPERTY_ID = process.env.UPPSOLUT_PROPERTY_ID;

async function api(path, init = {}) {
  const res = await fetch(`${BASE}${path}`, {
    ...init,
    headers: { Authorization: `Bearer ${KEY}`, "Content-Type": "application/json",
      ...(init.headers ?? {}) },
  });
  const body = await res.json();
  if (!res.ok) throw Object.assign(new Error(body.error),
    { code: body.code, status: res.status, details: body.details });
  return body;
}

// Property page
const { property } = await api(`/properties/${PROPERTY_ID}`);

// Search
const availability = await api(
  `/properties/${PROPERTY_ID}/availability?from=2026-03-10&to=2026-03-12`);
const options = availability.roomTypes.filter((rt) => rt.bookable);

// Price
const { quote } = await api(`/properties/${PROPERTY_ID}/quote`, {
  method: "POST",
  body: JSON.stringify({ checkIn: "2026-03-10", checkOut: "2026-03-12",
    roomId: options[0].id, adults: 2, children: 0 }),
});
```

```
});  
  
// Book  
const { booking } = await api(`/properties/${PROPERTY_ID}/bookings`, {  
  method: "POST",  
  headers: { "Idempotency-Key": crypto.randomUUID() },  
  body: JSON.stringify({  
    checkIn: "2026-03-10", checkOut: "2026-03-12", roomTypeId: options[0].id,  
    adults: 2, children: 0,  
    guest: { firstName: "Ada", lastName: "Lovelace",  
            email: "ada@example.com", phone: "+44 7700 900000" },  
  }),  
});  
console.log(booking.confirmationNo);
```

A machine-readable OpenAPI 3.1 description of this API is available alongside this guide as `website-api.openapi.yaml`, suitable for generating clients or importing into API tools.

APPENDIX A

Error codes

Every error response carries one of these codes. Switch on the code, show your own message to the guest, and log `error` and `details`.

Status	Code	Meaning and what to do
401	MISSING_API_KEY	No key header. Fix the integration.
401	INVALID_API_KEY	Unknown, revoked or expired key. Check the Hub; rotate or create a new key.
400	VALIDATION	Body or query failed validation. Map <code>details</code> onto form fields.
400	INVALID_DATES	Dates not <code>YYYY-MM-DD</code> , or check-out not after check-in.
400	ARRIVAL_IN_PAST	Check-in is before the property's business date. Rebuild the picker off <code>businessDate</code> .
400	TOO_FAR_AHEAD	Check-out is beyond the booking window set in the Hub.
400	MIN_STAY	Fewer nights than the property's minimum.
400	STAY_TOO_LONG	More than 62 nights requested.
400	INVALID_OCCUPANCY	Adults under 1, or adults plus children over the room type's maximum.
404	PROPERTY_NOT_FOUND	Unknown id, or a property the key does not cover. Never a 403, so keys cannot be used to enumerate properties.
404	ROOM_TYPE_NOT_FOUND	Not a sellable room type of this property.
404	BOOKING_NOT_FOUND	Lookup: no match for confirmation number and email under this key.
409	BOOKING_DISABLED	Online booking is off, or no rate plan is configured, in the Hub.
409	NO_RATE	One or more nights have no price. Ask the property to fill the Price Calendar.
409	STOP_SALE	The property has closed one or more of the dates.
409	SOLD_OUT	No room of that type is free for every night. Send the guest back to the dates step.
409	MEAL_PLAN_NOT_OFFERED	You sent a meal plan but the property does not let guests choose one.
400	MEAL_PLAN_NOT_FOUND	Not an active meal plan of this property.
409	ADD_ONS_NOT_OFFERED	You sent extras but the property does not sell them online.

Status	Code	Meaning and what to do
400	ADD_ON_NOT_FOUND	An id is not an extra this property sells separately.
409	IDEMPOTENCY_CONFLICT	The <code>Idempotency-Key</code> was already used for a different property.
400	BOOKING_REJECTED	The reservation service refused for another reason; <code>error</code> explains.
500	INTERNAL_ERROR	Server-side failure. Safe to retry a GET; retry a booking with the same <code>Idempotency-Key</code> .

APPENDIX B

Go-live checklist

Work through this with the administrator and the developer together before the booking page goes public.

In the Hub

- ☐ Property configured: rate plan chosen, meal plan, minimum stay, booking window, desk note.
- ☐ Headline, description, photos and policies entered and proofread.
- ☐ Price Calendar filled for the plan across the booking window. Check with the availability endpoint for any `price: null` nights.
- ☐ If extras are sold online: the right ones are ticked under *Extras offered through the APIs*, and each has a current price in Allocations.
- ☐ Key created for the exact properties needed; origins empty for server-to-server, or set to the real domains only.
- ☐ Someone owns key rotation and knows where to revoke.

On the website

- ☐ Key stored as a server secret; not present in any client bundle or repository.
- ☐ Date picker starts at `businessDate` and stops at the booking window.
- ☐ Booking flow: quote, then book with `Idempotency-Key` ; retry with the same key on timeout.
- ☐ `SOLD_OUT` , `STOP_SALE` , `NO_RATE` , `BOOKING_DISABLED` and `VALIDATION` all handled with guest-friendly messages.
- ☐ Prices labelled correctly for `pricesIncludeTaxes` ; the quote's `grandTotal` is the displayed price.
- ☐ Meal plans and extras read from the property each time, not hard-coded; the page copes with both lists being empty.
- ☐ Confirmation page and confirmation email show the confirmation number, dates, room type, total, check-in and check-out times, and policies.

Test together

- ☐ A test booking made on the site is found at the front desk under Reservations with the "Booked via website" remark and the desk note.
- ☐ The test booking is cancelled by the desk, and the room shows as available again on the site.
- ☐ A stop-sale set on a test date shows as closed on the site and refuses a booking.
- ☐ The key's "last used" and booking count update in the Hub.

SUPPORT

For help with the Hub or the API, contact your Uppsolut Stay administrator or the Uppsolut support desk. Include the property code, the key's name (never the key itself), the endpoint called, and the `code` from the error response.